

```

    }
    clearInterval(timer); (7)
    timer = null;
}

function onSuccess() {
    console.log("playAudio():Audio Success");
}

function onError(error) {
    alert('code: ' + error.code + '\n' +
        'message: ' + error.message + '\n');
}

function setAudioPosition(position) {
    document.getElementById('audio_position').
innerHTML = position; (8)
}

</script></head><body> <h2> Playing Audio</h2>
<input type="button" value="Play Audio"
onclick="playAudio('file:///android_asset/www/01..waka.waka.
mp3');"></input><br> (9)
<input type="button" value="Pause Audio"
onclick="pauseAudio();"></input><br>
<input type="button" value="Stop Audio"
onclick="stopAudio();"></input><br><br>
<p id="audio_position">Interval</p> (10)
</body> </html>

```



Note: *media.html* illustrates the usage of the play, pause and stop methods. The methods *media.startRecord()* and *media.stopRecord()* can be used similarly.

media.html will be displayed as shown in Figure 2. Here is the explanation of the code given above:

1. On clicking the button *Play audio*, the method 'playAudio()' with the path of the .mp3 file will be called and src will be replaced by the mp3 file. In case of success or failure, the methods 'mediaSuccess()'/'mediaError()' will be called, respectively.
2. The audio.play() method will be invoked and audio will start playing.
3. This will check for the timer and if the timer is null, the setInterval function will be called and the audio's current position will be fetched.
4. If the fetched position is greater than -1 then the 'setAudioPosition(position) + " sec"' method will be called, and in Line 8, the setAudioPosition() method will fetch the element by ID and replace its text with the fetched current position.
5. pauseAudio() will pause the already playing audio, and the interval which is displayed below will be paused.

6. stopAudio() will stop the audio and will call clearInterval().
7. clearInterval() will clear the interval displayed below the buttons.
8. This aspect has been already covered in Point 4.
9. On clicking this button, the path of the audio file to be played needs to be given as a parameter to the playAudio() method.
10. This is an element to display the position of the running audio file in terms of seconds. It's used when the element's text is replaced by the position in the setAudioPosition() method.

Playing Audio

Play Audio
Pause Audio
Stop Audio

39.192 sec

Figure 2: Media

Notifications

Device notifications are used to send alerts to the user or to ask for user input, and so on. These include visual, audio and vibration alerts. To use device alerts in your hybrid app, try the Notification API.

Usage: To alert the user to various events.

The methods for using this API are:

1. navigator.notification.alert(): To show alerts to the user for critical battery, low memory space, etc.
2. navigator.notification.confirm(): To ask the user for some kind of permission.
3. navigator.notification.prompt(): To prompt the user to provide some input.
4. navigator.beep(): To send audio alerts in the form of beeps.
5. navigator.vibrate(): To send a vibration alert to give the users some message or inform them about an event.

The arguments to pass through these methods are:

1. navigator.notification.alert ('message', alertCallback, 'title', 'buttonLabel'): Where a message will be displayed as an alert message, and alertCallback will be the method that gets called when the *Alert* button gets clicked; *title* will be the title of the alert box and *buttonLabel* will be the name of the *Alert* button.
2. navigator.notification.confirm ('message', confirmCallback, 'title', 'btnLabel1, btnLabel2,...'): All arguments are similar to that of the *Alert* box but, in this case, the buttons can be more than one, to provide multiple options to the user.
3. navigator.notification.prompt ('message', confirmCallback, 'title', 'btnLabel1, btnLabel2,...'): The arguments are the same as in the 'Confirm' method.
4. navigator.beep(times): The argument 'times' decides how many times the beep should ring.
5. navigator.vibrate (milliseconds): This depicts for how many seconds the device will vibrate.